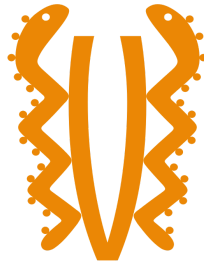


CONCLUSIONES



Autor:
WILMER FERNANDO RAMOS ORTEGA

Curso:
Seguridad de la Información

UNIVERSIDAD EL BOSQUE
PROGRAMA DE INGENIERÍA DE SISTEMAS
FACULTAD DE INGENIERÍA
Bogotá, 2026

Introducción:

En el desarrollo de aplicaciones web modernas existe una premisa arquitectónica no negociable: el cliente (navegador web) es un entorno hostil y no confiable. Cualquier validación, verificación de contraseñas, control de accesos o cálculo criptográfico de autenticación que resida en el lado del cliente (HTML, CSS, JavaScript) es susceptible a ser inspeccionado, alterado o completamente ignorado por un atacante.

Este documento presenta la justificación técnica, conceptual y práctica de por qué los controles críticos de seguridad deben residir y ejecutarse de forma exclusiva en el servidor (backend).

1. El Navegador es un Entorno de Ejecución Bajo Control del Usuario (Cliente Hostil)

A diferencia del servidor, cuyos procesos y memoria están protegidos dentro de la infraestructura del proveedor o de la empresa, el código que corre en el navegador del usuario se ejecuta en el dispositivo de este. Esto conlleva dos implicaciones inmediatas:

Inspección Total del Código Fuente: Todo el código HTML y JavaScript es entregado en texto claro al navegador. Incluso con técnicas de ofuscación o minificación, el flujo de ejecución puede ser desensamblado y analizado mediante las herramientas de desarrollo (DevTools).

Manipulación Dinámica en Tiempo de Ejecución (Runtime Tampering): A través de la consola de desarrollador del navegador, cualquier atacante puede modificar variables en memoria (por ejemplo, cambiar `usuario.esAdmin = false` a `true`), sobrescribir funciones de validación (por ejemplo, redefinir `validarPassword = () => true`), alterar los atributos del DOM (eliminar atributos `required`, `pattern`, `disabled` o límites de caracteres), y establecer breakpoints para alterar el flujo de las sentencias condicionales (`if/else`).

2. Evasión Completa del Navegador Mediante Peticiones HTTP Directas

El argumento definitivo contra la seguridad en el cliente radica en el propio protocolo HTTP: el servidor no sabe si una petición proviene de un navegador legítimo o de una herramienta automatizada.

Un atacante no requiere abrir un navegador web para interactuar con una aplicación. Puede utilizar clientes HTTP por línea de comandos como `cURL`, herramientas de prueba como Postman o scripts en Python (`requests`), Go o Node.js. Al enviar una solicitud directa vía `POST /login` o `POST`

/api/crypto/analyze, ninguna línea de código JavaScript del frontend se ejecuta. Por consiguiente, si las reglas de validación (complejidad de contraseñas, conteo de intentos fallidos, verificación de roles) estuvieran en el frontend, el atacante las evadiría con una sola línea de comando en su terminal:

Ejemplo: Un ataque automatizado ignora completamente cualquier JS del navegador

```
curl -X POST http://servidor.com/login -H "Content-Type: application/json" -d '{"username": "admin", "password": "123"}'
```

3. Vulnerabilidades del Hashing de Contraseñas en el Cliente

Intentar calcular el hash de una contraseña en el navegador antes de enviarla al servidor introduce graves fallas de diseño:

Exposición del Salt: Si el salt se genera en el cliente o es estático, queda visible para cualquier observador. Si se solicita previamente al servidor mediante un endpoint adicional, se duplica la superficie de ataque.

Ataque de "Pass the Hash" Web: Si el cliente hashea la contraseña con SHA-256 o MD5 y envía ese hash al servidor como si fuera la credencial definitiva, el hash resultante se convierte funcionalmente en la nueva contraseña en texto claro. Cualquier atacante que intercepte dicho hash (por ejemplo, mediante una fuga de datos previa) podrá autenticarse sin necesidad de conocer la contraseña original.

Poder de Cómputo No Regulado: Los algoritmos diseñados para contraseñas como BCrypt requieren un factor de trabajo (work factor o costo computacional, como $2^{10} = 1024$ iteraciones con salt criptográfico de 128 bits). Forzar este cálculo en clientes de bajos recursos (como teléfonos móviles antiguos) degrada la experiencia de usuario, mientras que en el backend se puede dimensionar y optimizar de manera controlada.

4. El Rol Real de las Validaciones en el Frontend: Usabilidad vs. Seguridad

¿Significa esto que las validaciones en el cliente deben eliminarse? No. Sin embargo, su propósito debe redefinirse estrictamente:

Frontend: Su función es exclusivamente la Usabilidad y Experiencia de Usuario (UX). Permite advertir al usuario si olvidó ingresar un campo, si el formato de correo carece de un @ o si la contraseña ingresada no cumple visualmente los requisitos mínimos, evitando esperas innecesarias de ida y vuelta a la red.

Backend: Su función es la Garantía y Control Perimetral de Seguridad. Es la única autoridad con capacidad de autenticar, autorizar, sanitizar entradas contra inyecciones SQL/NoSQL/XSS, limitar la tasa de peticiones (Rate Limiting) y proteger la integridad del sistema.

5. Implementación en este Laboratorio: Defensa en Profundidad

Para dar cumplimiento estricto a las mejores prácticas de la industria y a la Guía de Laboratorio (Universidad El Bosque):

Esquema Relacional RBAC en Backend: Las identidades y permisos residen en tablas normalizadas (roles, usuarios) dentro de una base de datos gestionada con consultas preparadas parametrizadas que impiden la inyección SQL.

Hashing Unidireccional con BCrypt (Costo 10): Las contraseñas nunca se almacenan en texto claro. BCrypt concatena un salt aleatorio de 128 bits y ejecuta 1024 rondas de cifrado EksBlowfish. La verificación asíncrona mediante `bcrypt.compare` opera exclusivamente en el servidor.

Middleware Perimetral Anti-Fuerza Bruta: Las solicitudes se filtran antes de alcanzar la lógica de la aplicación. Al quinto intento fallido desde una misma dirección IP, el middleware corta la comunicación devolviendo un código de estado HTTP 429 Too Many Requests, protegiendo la base de datos y los ciclos de CPU del servidor.

6. Conclusión Final

"La seguridad en el lado del cliente es una ilusión, la seguridad en el lado del servidor es una necesidad."

Cualquier sistema que delegue decisiones de seguridad al navegador es inherentemente vulnerable. La arquitectura implementada en este laboratorio garantiza que, incluso si un atacante elude por completo la interfaz HTML/JS o modifica los scripts del cliente mediante DevTools o cURL, el backend mantendrá inalteradas sus políticas de autenticación, hashing seguro y mitigación de fuerza bruta.